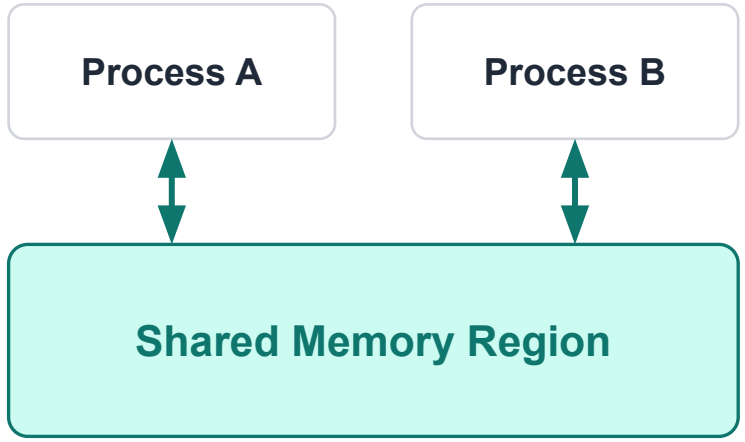
Shared Memory Based Data Exchange System

Efficient Inter-Process Communication with Data Consistency



Team No: 11

Section: 1

Team Members :

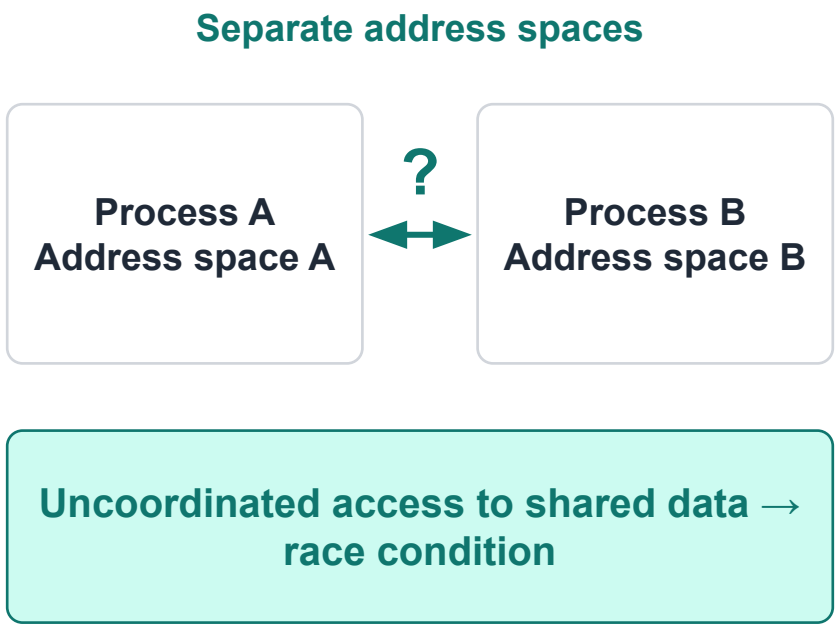
M. Bala Pranitha – 2520030051

S. Manvi Vaishanvi – 2520030326

D. Rohini Tanmayi – 25200300349

Problem Statement

- Multiple processes often need to exchange data efficiently.
- Each process normally has its own memory address space.
- Shared data accessed without coordination can cause race conditions and inconsistent results.
- A reliable mechanism must coordinate reading and writing.



CORE PROBLEM

“How can multiple processes exchange data efficiently while ensuring that shared data remains consistent?”

Project Objectives

01

Implement inter-process communication using shared memory.

02

Enable a producer process to write messages and a consumer process to read them.

03

Synchronize access using process-shared semaphores.

04

Prevent unread messages from being overwritten.

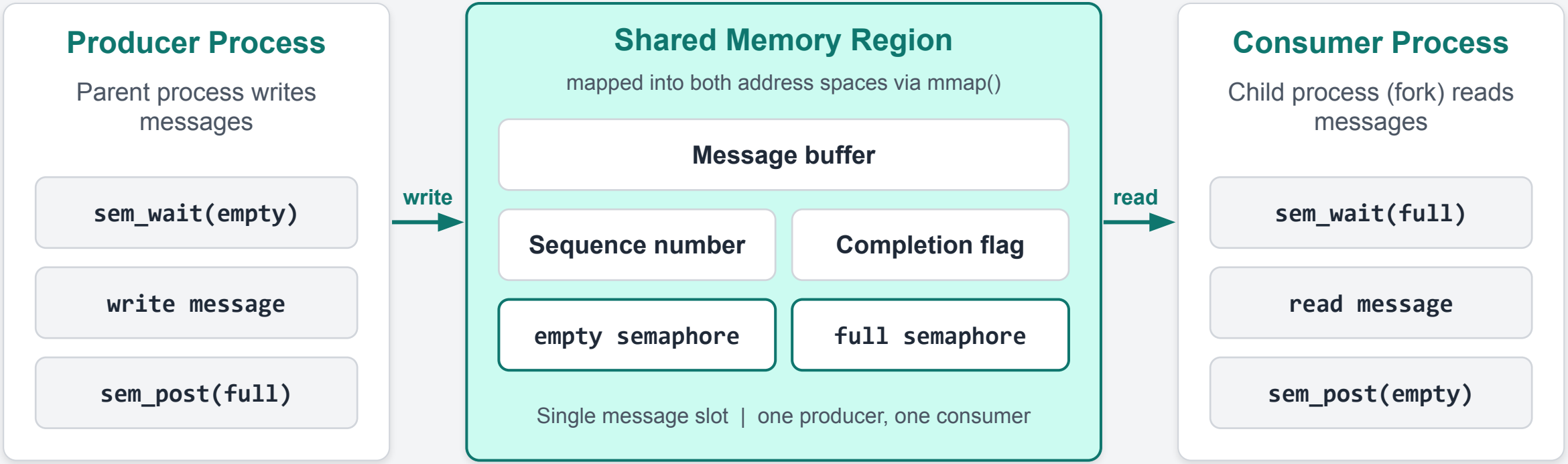
05

Validate message order and content through testing.

06

Demonstrate a working prototype on Ubuntu Linux.

System Architecture



- The producer writes a message into the shared memory.
- The consumer reads the message from the same mapped region.
- Semaphores (empty, full) coordinate when writing and reading may occur.
- A completion flag signals the end of communication.

Technologies and Tools

C Programming Language

Implements process creation, shared data structures, and synchronization.

Ubuntu Linux

Provides process management and POSIX inter-process communication facilities.

POSIX Shared Memory

`shm_open()` creates or opens the object

`ftruncate()` sets the object size

`mmap()` maps it into the address space

POSIX Semaphores

`sem_init()` initializes process-shared semaphores

`sem_wait()` waits for permission to proceed

`sem_post()` signals an operation has completed

Other Tools

`fork()` creates the consumer child process

`GCC` compiles the C program

`Git / GitHub` version control and code hosting

Implementation Approach

1. **`shm_open()`**
   Create the shared memory object.

2. **`ftruncate()`**
   Allocate the required size.

3. **`mmap()`**
   Map the memory into the address space.

4. **Initialize**
   Set empty = 1 and full = 0.

5. **`fork()`**
   Create the consumer process.

6. **Producer**
   Waits for an empty slot, writes a message, signals full.

7. **Consumer**
   Waits for a message, reads it, signals empty.

8. **Finish**
   Send completion signal and clean up resources.

Producer

```
wait(empty)
write(message)
post(full)
```

Consumer

```
wait(full)
read(message)
post(empty)
```

Maintaining Data Consistency

Without synchronization, the producer could overwrite an unread message, or the consumer could read before a message is ready.

`empty` semaphore

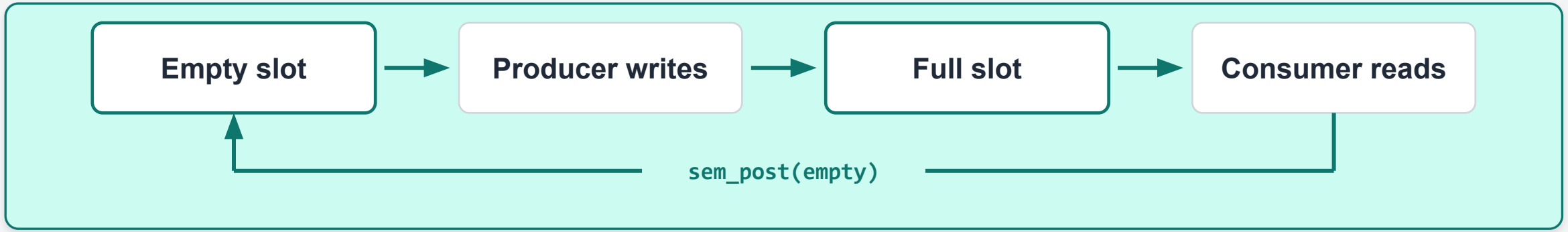
Initial: 1

- Indicates the single shared slot is available for writing.
- The producer waits on it before writing.
- The consumer signals it after reading.

`full` semaphore

Initial: 0

- Indicates a message is available for reading.
- The producer signals it after writing.
- The consumer waits on it before reading.



Semaphores coordinate the two processes and prevent the producer from overwriting unread data in this single-slot design.

Testing, Validation and Results

Test 1: Compilation
Compile the C source code using GCC.

PENDING

Test 2: Message exchange
Run the prototype and verify that the producer writes five messages.

PENDING

Test 3: Data consistency
Consumer receives sequence numbers 1, 2, 3, 4 and 5 with matching message contents.

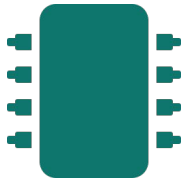
PENDING

Test 4: Completion
Verify that the consumer exits after receiving the completion signal.

PENDING

Test 5: Repeated execution
Run the program multiple times and check for normal completion.

PENDING



[Insert Ubuntu terminal screenshot]
Show producer output, consumer output, and successful completion after running the program.

Expected result (verify by running)
Consumer receives all five messages in sequence, receives the completion signal, and terminates.

Applications and Future Enhancements

Applications

- Communication between cooperating local processes.
- Producer-consumer systems.
- Operating-system and systems-programming demonstrations.
- Low-overhead data exchange between applications on the same machine.

Project Impact

- Demonstrates inter-process communication.
- Illustrates synchronization and race-condition prevention.
- Foundation for more advanced shared-memory systems.

Current Limitations

- One producer and one consumer.
- One shared message slot.
- No crash recovery or persistent message storage.
- No graphical monitoring interface.

Future Enhancements

- Circular buffer supporting multiple queued messages.
- Multiple producers and consumers.
- Robust error handling and crash recovery.
- Performance measurement and comparison with other IPC mechanisms.
- A monitoring interface for process activity.

Conclusion

- Implemented a C-based shared-memory communication prototype.
- Used POSIX shared memory for data exchange.
- Used process-shared semaphores to coordinate producer and consumer operations.
- Designed tests for message order, correctness, and normal termination.
- Established a foundation for future multi-process communication enhancements.

Thank You

Questions and Answers

Team: [Team Name] | Team No. 11 | Section 1

Members: M. Bala Pranitha, S. Manvi Vaishanvi, D. Rohini Tanmayi

GitHub: [https://github.com/your-repo-link]

